Cadra API — Demo Card

One-page cheat sheet: 6 endpoints that tell the whole product story in under 10 minutes. Every block is copy-paste runnable against staging.

Setup (run once)

```
API=https://staging.app.cadra.ng
export CLIENT_SECRET='<from docs/api/.creds>'

TOKEN=$(curl -sX POST $API/api/v1/auth/token \
  -H 'Content-Type: application/json' \
  -d '{"client_id":"test_client","client_secret":"'"$CLIENT_SECRET"'"}' \
  | jq -r .data.access_token)

acurl() { curl -sS -H "Authorization: Bearer $TOKEN" "$@"; }
```

1. Auth — get a token

```
curl -sX POST $API/api/v1/auth/token \
  -H 'Content-Type: application/json' \
  -d '{"client_id":"test_client","client_secret":"'"$CLIENT_SECRET"'"}'
```

Returns: `Bearer` token, 1-hour TTL, plus the granted scopes. **Talking point:** every other call sends this token; scopes restrict what each integration can do.

2. Create a service request (full ticket in one call)

```
acurl -X POST $API/api/v1/requests \
  -H 'Content-Type: application/json' \
  -H 'Idempotency-Key: demo-card-001' \
  -d '{
    "type_id": 1,
    "category_id": 5,
    "priority": "3",
    "request_text": "<p>UPS in Bldg A failed overnight</p>",
    "nrs_asset_id": 124
  }'
```

Returns: `INCIDENT-MMYYYY-NNNNNNN` reference, initial stage, assignee slot, available routes. **Talking point:** one API call → a full ITSM ticket with auto-generated reference, ready for the support desk.

3. Move an asset (side-effect demo — creates an audit row)

```
acurl -X POST $API/api/v1/assets/124/move \
  -H 'Content-Type: application/json' \
  -d '{"to_location_id": 905, "reason": "Q2 audit reshuffle"}'
```

Returns: updated asset + a new `nrs.asset.transfer` audit row (`{from_location, to_location, reason, created_at}`). **Talking point:** the API doesn't just write a field — it records the history. Compliance audits get a chain of custody for every asset move.

4. Read the same data the UI reads

```
acurl $API/api/v1/fm/dashboard | jq '.data.kpis'
```

Returns: `{open, overdue, escalated, in_progress, resolved_today}` plus SLA panel + map points. **Talking point:** open the FM dashboard in the browser side-by-side — this JSON **is** the chart payload. One source of truth, no API/UI drift.

5. Inbound integration (SAP employee sync)

```
acurl -X POST $API/api/v1/integrations/sap/employee-sync \
  -H 'Content-Type: application/json' \
  -d '{
    "ir_number": "FIRS-12345",
    "userId": "adaobi.eze@firs.gov.ng",
    "firstName": "Adaobi",
    "lastName": "Eze",
    "department": "ICT",
    "jobTitle": "Senior Manager",
    "status": "active",
    "officeCode": "HQ"
  }'
```

Returns: `{action: "created"|"updated", employee_id, partner_id}`. **Talking point:** SAP pushes hire/move/terminate events; Cadra keeps `hr.employee` in sync. Idempotent — replays don't duplicate.

6. Self-documenting (the closing flex)

```
# Live count of registered endpoints
acurl $API/api/v1/openapi.json | jq '.paths | keys | length'   # → 39 unique paths, 45 method×path combos

# Then in a browser:
open $API/api/v1/docs
```

Returns: the OpenAPI 3.0 spec, regenerated **at every request from the live route registry**. Click "Authorize" on `/api/v1/docs`, paste the JWT, every endpoint becomes interactive. **Talking point:** the spec can't drift from the code — it **is** the code, reflected. Auditors love this.

Error responses worth showing on demand

```
# 401 — no token
curl -sS $API/api/v1/assets | jq '.error'
# { "code": "unauthorized", "message": "Missing bearer token", ... }

# 400 — bad input
acurl -X POST $API/api/v1/requests -H 'Content-Type: application/json' -d '{}' | jq '.error'
# { "code": "invalid_request", "message": "`type_id` is required", ... }

# 404 — unknown resource
acurl $API/api/v1/assets/99999999 | jq '.error'
# { "code": "not_found", "message": "Asset 99999999 not found", ... }

# 400 — invariant violation (workflow callback with unknown outcome)
acurl -X POST $API/api/v1/integrations/workflow/callback \
  -H 'Content-Type: application/json' \
  -d '{"reference":"INCIDENT-052026-0000002","outcome":"bogus"}' | jq '.error'
# { "code": "invalid_request", "message": "Unknown outcome 'bogus'",
#   "details": { "allowed_outcomes": ["approved","rejected","queried"] } }
```

Pocket facts (slide-friendly)

- **45** endpoints across **3 core resources** (Assets · Locations · Requests) + dashboards + integrations
- **One envelope** shape, every response
- **13 scopes**, granular per-integration
- **JWT, HS256, 1h TTL** — issued by `/auth/token` from client credentials
- **Cursor pagination** + `Idempotency-Key` on every create endpoint
- **OpenAPI 3.0** spec auto-generated, Swagger UI at `/api/v1/docs`
- **Same compute path** as the Odoo webclient — no UI/API drift possible